



UFO SHUTTER SYSTEM

UFO Shutter System Documentation

**FIREBall UFO shutter reuse as a stand-alone 24 V
Arduino/Python-controlled shutter system**

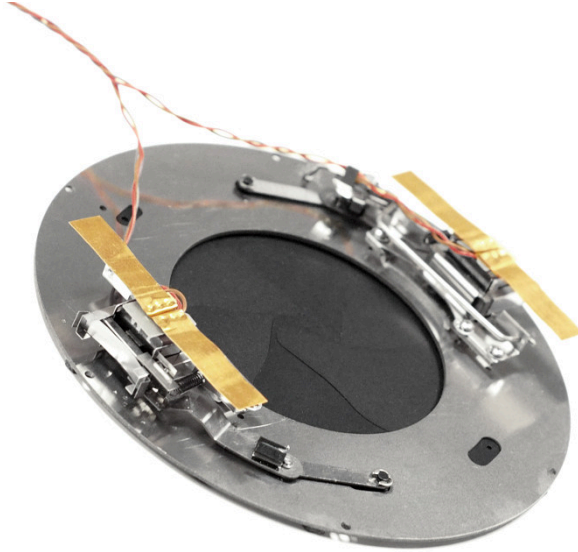
https://github.com/afham-b/ufo_shutter/

2026-09-27

Table of Contents

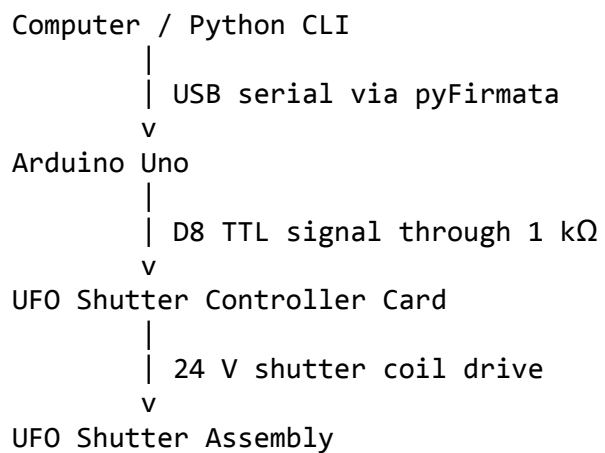
1. Overview
2. Safety Notes
3. Hardware
4. Single-Shutter Wiring
5. Dual-Shutter Relay Wiring
6. Capacitors, Snubbers, and Noise Mitigation
7. Software Setup
8. Running the Controller Script & Commands
9. Timing and Flux Calibration
10. Troubleshooting
11. Rewiring
12. Repository Layout
13. Notes
14. TO-DO list

1. System Overview



This project repurposes the FIREBall UFO shutter assembly and its original UFO Shutter Controller card as a stand-alone shutter controller for lab use. The system keeps the original controller card responsible for all high-current shutter coil drives. The Arduino is used only as a USB-controlled TTL I/O interface.

High-level control path:



For the dual-shutter configuration, the original UFO controller still provides the 24 V shutter drive on terminals 7 and 8, but a relay module switches those two drive lines between Shutter A and Shutter B.

UFO Controller pins 7/8



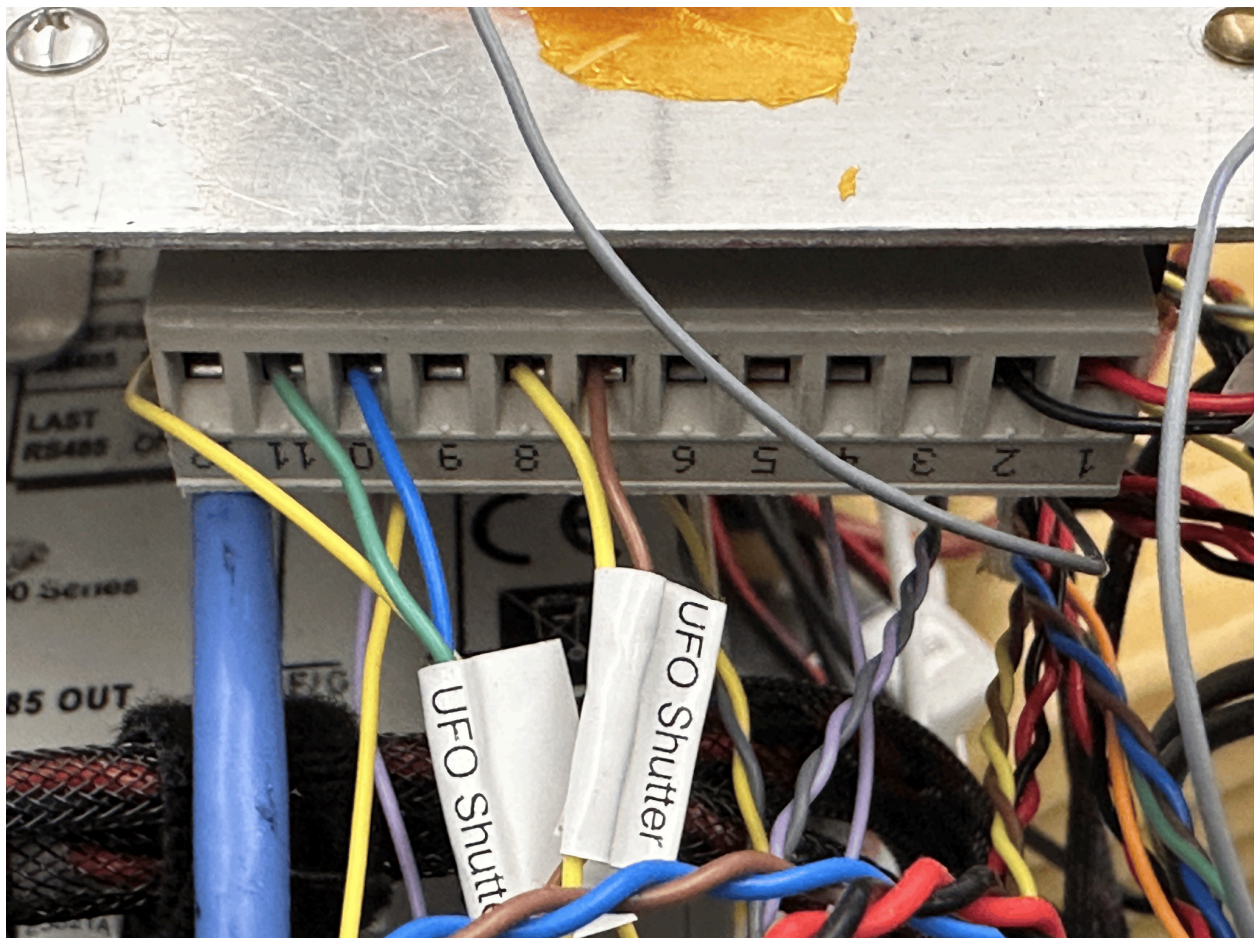
Dual relay switching board

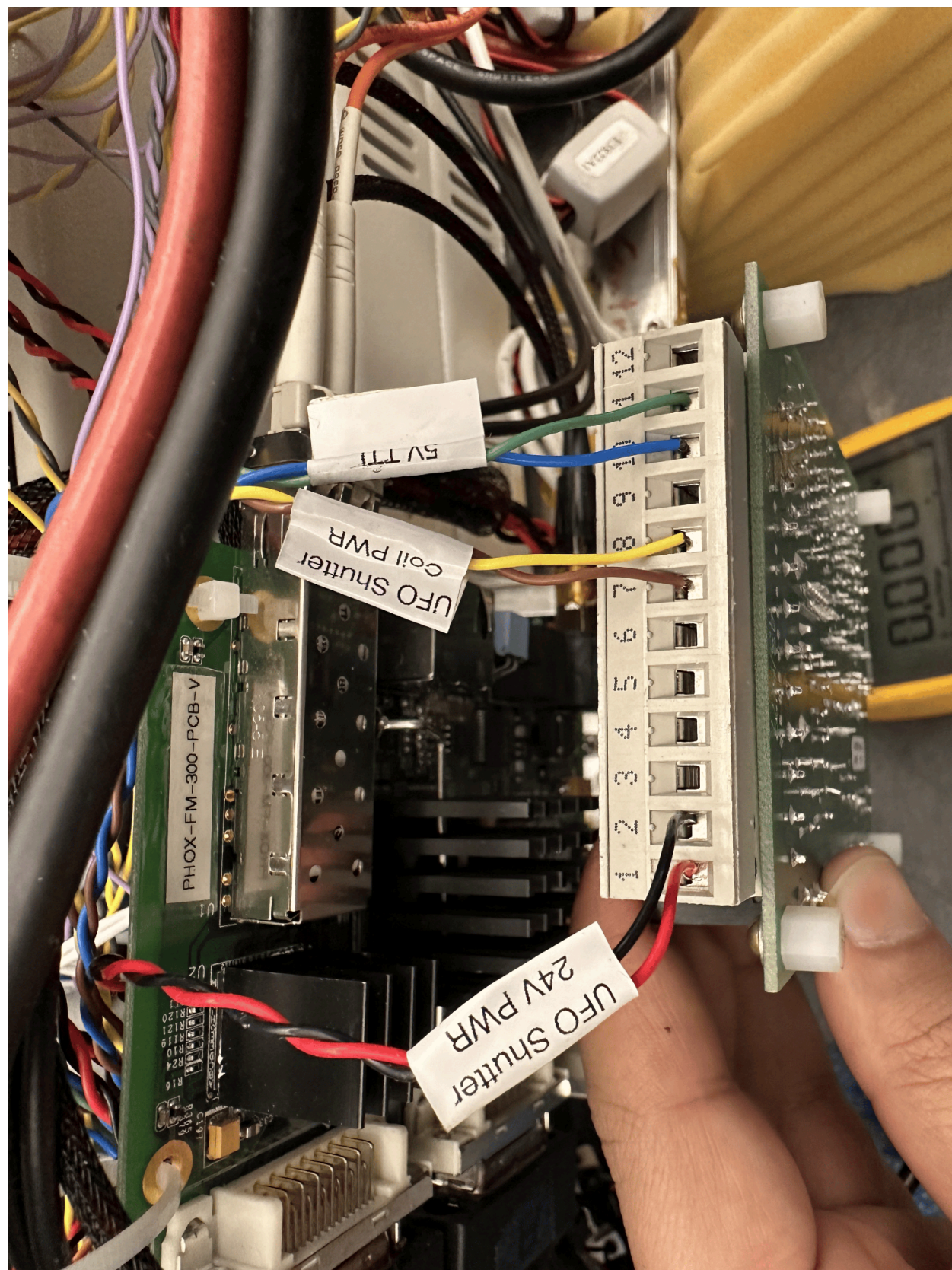
↓
Shutter A

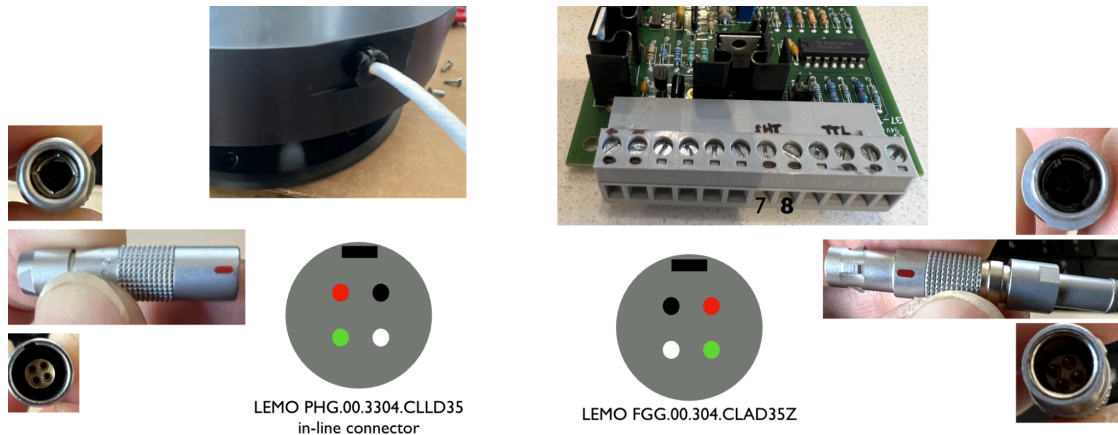
↓
Shutter B

Original FIREBall Context

Here are some of the original wiring setup within Fireball.







White cable between UFO (spectrograph tank) and shutter controller (GOBC): 4 wires - red, black, green, white

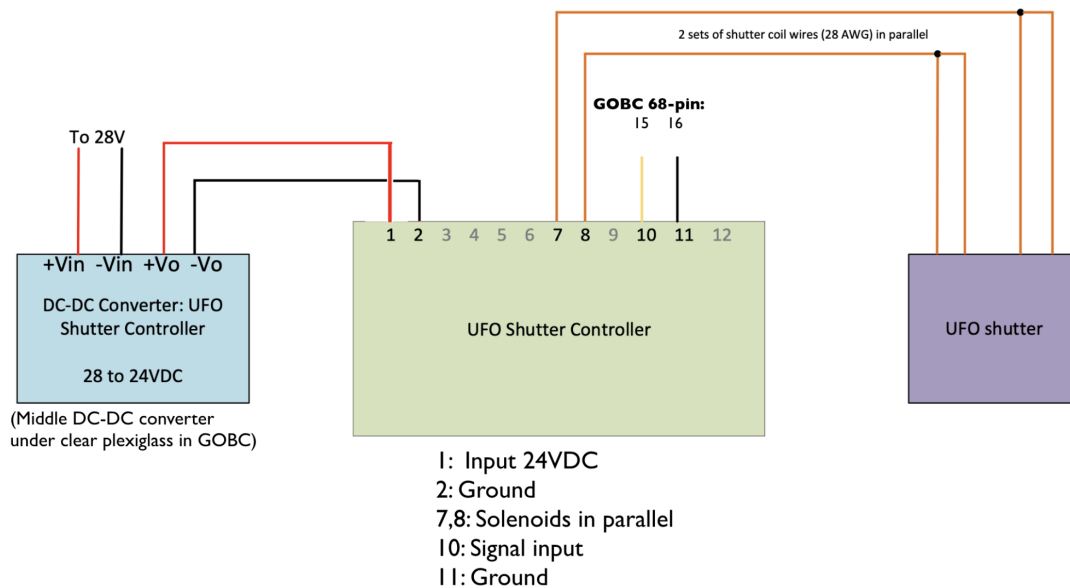
Flying leads with ferrules on the end of FGG connector:

Red and black wires are for shutter solenoid 1.

Green and white wires are for shutter solenoid 2.

Red and green go to pin 7, green and white go to pin 8 for solenoids wired in parallel.

UFO_SYS: UFO Shutter System



2. Safety Notes

This system contains low-voltage logic wiring and 24 V solenoid power wiring. Keep these domains separate.

- The Arduino must never be connected directly to 24 V.
 - The Arduino only drives the controller's TTL input and the relay module's logic input.
 - Disconnect 24 V power before moving shutter coil wiring, relay contact wiring, or controller terminal wiring.
 - Avoid switching the relay contacts while the shutter controller is actively energizing the coil. The safe switching logic opens/de-energizes first, waits, switches the relay, waits again, and optionally closes the newly selected shutter.
This is why delays are integrated into the code !
 - ***DO NOT connect or disconnect shutters while the 24V power supply is plugged in the controller box.***
 - The 680 μ F capacitor on the relay coil supply may remain charged briefly after power-off.
 - Verify terminal numbers and connector pinouts before applying power.
 - If the Arduino disconnects, resets, or computer reports Device not configured, treat it as a noise/EMI symptom until proven otherwise.
-
-

3. Hardware Inventory

Required for single-shutter operation

- UFO shutter assembly, two-coil solenoid type:
 - <https://www.uniblitz.com/product-category/optical-shutters/cs/>
- UFO Shutter Controller card - V880 Shutter card
 - Manual link: [Download](#)
 - Purchase link: <https://www.uniblitz.com/products/v880-shutter-driver/>
 - 3D model: [Download](#)
- White LEMO shutter cable between UFO shutter and controller
- 24 V DC power supply
- Arduino Uno or compatible serial controller board
- USB type B cable
- 1 k Ω resistor in series with TTL signal input
- Jumper wires / terminal block wiring

Additional parts for dual-shutter relay switching

- 2-channel relay module with JD-VCC coil-side power input
 - External 5 V relay supply, recommended 5 V / 2 A
 - 680 μ F electrolytic capacitor, at least 10 V rating; 16 V used in current wiring
 - Two 0.1 μ F ceramic capacitors
 - x2 RC snubber modules across relay contacts
 - Second shutter assembly and connector wiring
-

4. Single-Shutter Wiring

Connector wiring correction note: Use the text table below as the authoritative mapping. The current verified parallel wiring is **Red + Green -> terminal 7** and **Black + White -> terminal 8**.

The single-shutter configuration is the simplest known-good setup.

24 V power

24 V PSU + -> UFO controller terminal 1

24 V PSU - -> UFO controller terminal 2

Shutter coil wiring

The original white LEMO cable has four wires:

Wire color	Coil assignment
Red	Solenoid 1
Black	Solenoid 1
Green	Solenoid 2
White	Solenoid 2

For parallel coil wiring:

Red + Green -> UFO controller terminal 7

Black + White -> UFO controller terminal 8

Always verify with continuity if a connector has been reworked.

Arduino TTL wiring

Arduino D8 -> 1 k Ω resistor -> UFO controller terminal 10

Arduino GND -> UFO controller terminal 11

The 1 k Ω resistor is a series protection resistor between Arduino D8 and the controller TTL input.

Logic polarity

The current working polarity is:

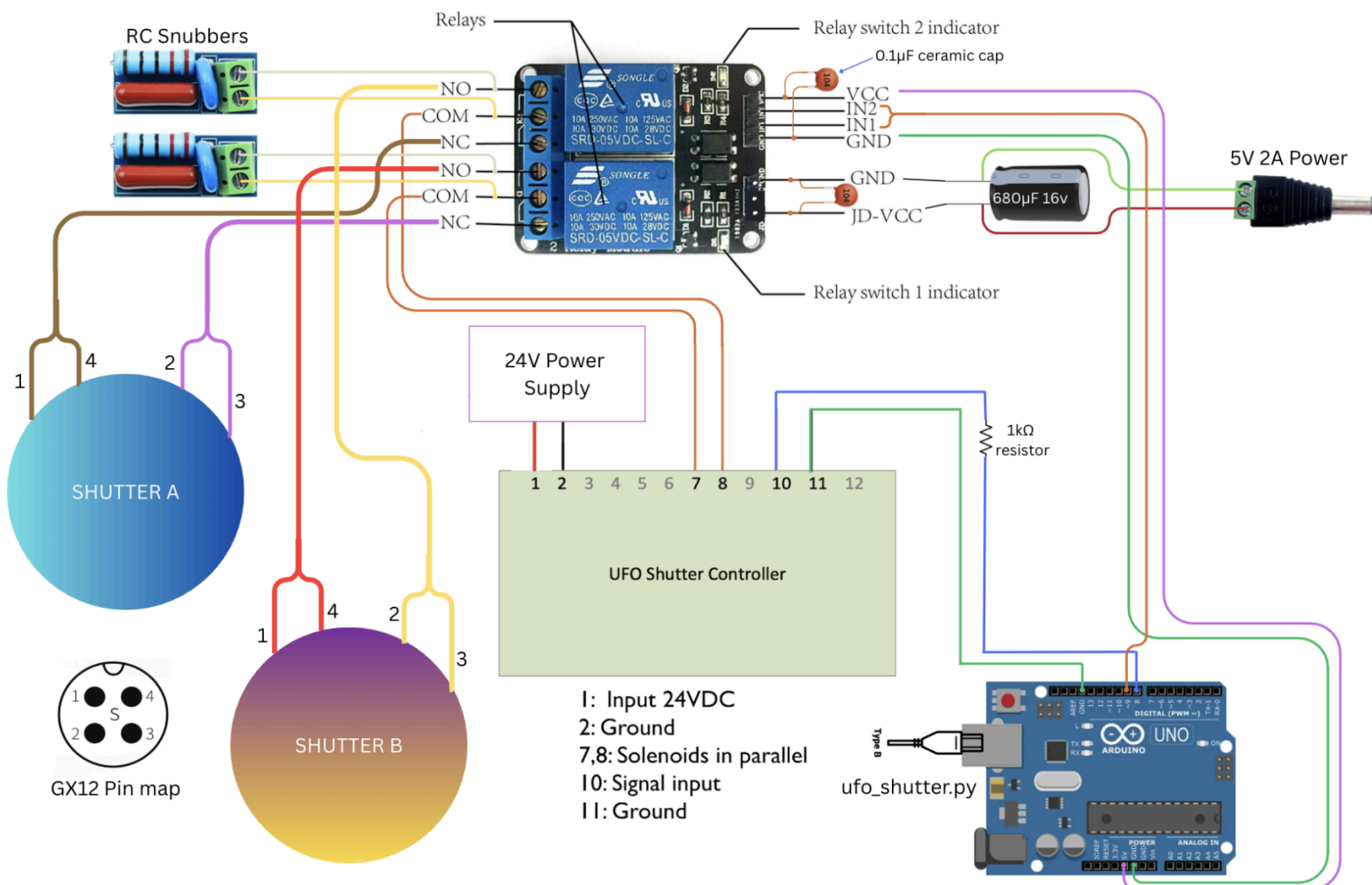
OPEN_STATE = 0
CLOSED_STATE = 1

In this wiring, OPEN_STATE = 0 means the shutter is open/de-energized, and CLOSED_STATE = 1 means the controller drives the shutter closed/energized. If a future controller or wiring revision behaves inverted, swap these two constants.

To run the shutter system in single shutter mode, simply plug the shutter's input GX-12 4-pin wire into the shutter A terminal of the shutter controller box. The relay defaults to shutter A on startup, and is what the relay's initial state is.

Alternatively, if the relay unit fails to energize, or mechanically fails, you may also attempt plugging into Shutter B terminal on the controller box as well, if using the shutter A terminal does not work.

5. Dual-Shutter Relay Wiring



Dual-shutter relay wiring with snubbers and relay supply capacitors

The V880 shutter controller natively drives one shutter output channel. The dual-shutter setup uses a two-channel relay module to switch the controller's two solenoid output lines, terminals 7 and 8, between two shutter assemblies. Functionally, the relay module acts like a DPDT switch when both relay channels are driven together. Only one shutter can be selected and operated at a time.

This setup inherently comes with limitations. Both shutter may not be operated at the same time, only one shutter may be selected and controlled, and the code allows the user to switch between which one to operate.

Relay contact side

The relay module terminal order is:

NO - COM - NC

for each relay channel.

The two controller solenoid outputs go to the relay COM contacts:

UFO controller terminal 7 -> relay channel 1 COM

UFO controller terminal 8 -> relay channel 2 COM

Shutter A is wired to the normally closed contacts so it is selected by default when the relays are off:

Relay channel 1 NC -> Shutter A coil side 1

Relay channel 2 NC -> Shutter A coil side 2

Shutter B is wired to the normally open contacts so it is selected when the relays are energized:

Relay channel 1 NO -> Shutter B coil side 1

Relay channel 2 NO -> Shutter B coil side 2

Relay logic side

Arduino D9 -> relay IN1 and relay IN2 tied together

Arduino 5V -> relay logic VCC

Arduino GND -> relay logic GND

Both relay channels switch together because both solenoid lines must move to the same shutter, thus allowing IN1 and IN2 to be tied together.

Relay coil power side

The relay itself use a separate 5 V supply for relay coil power:

External 5 V PSU + -> relay JD-VCC
External 5 V PSU - -> relay coil GND

The current wiring uses a 5 V / 2 A supply.

Do not bridge JD-VCC to VCC on the relay module. Leave the JD-VCC jumper removed when using a separate relay coil supply.

Relay logic polarity

The current relay module is active-low:

RELAY_ON = 0 # relays energized, LEDs on, COM -> NO, Shutter B selected
RELAY_OFF = 1 # relays off, COM -> NC, Shutter A selected

Therefore:

Selection	Arduino D9 / relay state	Contact path	Selected shutter
Shutter A	RELAY_OFF = 1	COM -> NC	A
Shutter B	RELAY_ON = 0	COM -> NO	B

6. Capacitors, Snubbers, and Noise Mitigation

The relay system can inject noise into the Arduino or USB link, as it switches. The following components reduce relay coil dips, contact arcing, and serial disconnects. While the power of the relay, arduino and 24v shutter card are separate, there is some back EMF that can cause the arduino's voltage to spike and shutdown the serial controller board and make the script unresponsive, and the user will see a communication error with the board.

It is also important to note that the noise mitigation and snubbers are placed so that they don't dampen any spikes or signals the shutter card sends to the shutters solenoids, as those solenoids expect this specific signal which has that initial transient period.

µF electrolytic on relay coil supply

Place the 680 µF capacitor across JD-VCC and relay coil ground:

680 µF electrolytic positive leg -> JD-VCC
680 µF electrolytic negative leg -> relay coil GND / external 5 V PSU negative

This capacitor supports the relay coil supply during switching.

0.1 μ F ceramic on relay coil supply

Place one 0.1 μ F ceramic capacitor in parallel with the 680 μ F capacitor:

0.1 μ F ceramic leg 1 -> JD-VCC

0.1 μ F ceramic leg 2 -> relay coil GND

Ceramic capacitors are non-polarized.

0.1 μ F ceramic on relay logic supply

Place the second 0.1 μ F ceramic capacitor across the relay logic supply:

0.1 μ F ceramic leg 1 -> relay logic VCC

0.1 μ F ceramic leg 2 -> relay logic GND

This helps keep logic-side noise from disturbing the Arduino reference.

RC snubbers on relay contacts

Place RC snubber modules across the relay contacts that switch the 24 V shutter lines.

For a shutter path on NO:

Snubber wire 1 -> NO

Snubber wire 2 -> COM

For a shutter path on NC:

Snubber wire 1 -> NC

Snubber wire 2 -> COM

Do not place large capacitors across the UFO controller shutter output; the controller may be shaping the solenoid drive waveform.

7. Software Setup

Arduino firmware

For Python control, users need to install Arduino IDE

If the board has been reset or replaced, (the current board in the controller box has this already loaded) user will need to upload StandardFirmata to the Arduino:

Arduino IDE -> File -> Examples -> Firmata -> StandardFirmata

After upload, the Arduino acts as a USB I/O board controlled by Python.

Python environment

From the repository folder:

```
python -m venv .venv
source .venv/bin/activate
python -m pip install --upgrade pip
pip install pyfirmata pyserial
```

- i. Once package is installed, in terminal: `pip show pyFirmata`
- ii. Go to location of package → open `pyfirmata.py`
 1. ~line 185: Replace `inspect.getargspec` with `inspect.getfullargspec` and SAVE

Serial ports

Code has implemented an auto port finder for win/mac/linux to identify and connect to the board, but if this fails, User may need to update this variable's value in the `ufo_shutter.py` script. Examples:

```
DEFAULT_PORT = 'COM3'           # Windows
DEFAULT_PORT = '/dev/ttyACM0'   # Linux
DEFAULT_PORT = '/dev/cu.usbmodem101' # macOS Arduino Uno example
DEFAULT_PORT = '/dev/cu.usbserial-10' # macOS USB-serial board example
```

On macOS, list likely devices:

```
ls /dev/cu.usb*
```

If you are unsure of which port, open the Arduino IDE, and see the ports dropdown list to see which port the board is connected to on the device once it has been plugged in.

8. Controller Script: `ufo_shutter.py`

The main Python script connects to the Arduino, configures the shutter TTL pin and relay-select pin, and provides an interactive CLI.

Main pin assignments

```
SHUTTER_PIN_NUM = 8 # Arduino D8 -> UFO controller TTL input through 1 kΩ
SELECT_PIN_NUM  = 9 # Arduino D9 -> relay IN1 and IN2
```

The pins correlate to the wires plugged into the arduino pins, if the pin locations are changed, these values will need to be updated.

Important constants

```
OPEN_STATE = 0
CLOSED_STATE = 1
```

```
RELAY_ON  = 0
RELAY_OFF = 1
```

Running the controller

Activate the virtual environment:

```
source .venv/bin/activate
```

Run with default port hardcoded:

```
python ufo_shutter.py
```

Run with explicit port to pass it a port if you don't want to hardcode it:

```
python ufo_shutter.py /dev/cu.usbserial-10
```

CLI commands

Command	Meaning
o	Open selected shutter
c	Close selected shutter
p <ms>	Pulse for target effective exposure using no compensation
p <ms> o	Pulse command for <ms> with compensation added in.
ra	Safely select Shutter A, relay OFF, COM -> NC
rb	Safely select Shutter B, relay ON, COM -> NO
rt	Relay toggle test, A/B switching
sw	Run predefined raw pulse sweep for timing/calibration
swo	Run predefined compensated pulse sweep
q	Quit, leave shutter open and relay on in current script defaults
qoff	Quit, leave shutter open and relay off
qcoff	Quit, leave shutter closed and relay off

Safe switching behavior

The safe relay switch routine induces delays in commands, which does the following:

1. Opens/de-energizes the selected shutter.
2. Waits for the driver and coil to settle.
3. Switches the relay state.
4. Waits for relay contacts to settle.
5. Optionally closes the newly selected shutter to establish a known state.

This prevents switching the relay contacts while the shutter coil is actively energized.

9. Fast Mode

Fast mode is intended for rapid shutter timing and flux-characterization work where the normal runtime command delays and relay-switching delays are not needed. It is an A-only mode: after initialization, the selector is a locked relay OFF / COM -> NC so only Shutter A can be used. Relay switching commands are disabled while fast mode is active.

Starting Fast Mode

Run the controller with:

```
python ufo_shutter.py --fast
```

An explicit port can still be supplied with --port or with the legacy positional-port form. Fast mode keeps the normal startup settling sequence, then reports READY when commands may be entered.

Fast CLI commands

Option	Meaning
--fast	Enable fast A-only mode, lock the relay selector OFF / NC, and remove normal runtime command/countdown delays.
--min-closed-ms <ms>	Minimum commanded-closed recovery time before the next opening pulse. Default is 200 ms and is currently provisional.
--sweep-gap-s <s>	Set the closed gap used by sw/swo. Fast mode still enforces the minimum closed recovery time.

Fast mode commands

Command	Fast mode behavior
o	Open Shutter A immediately; the normal DELAY_BEFORE_COMMAND is skipped.
c	Close Shutter A immediately; the normal DELAY_BEFORE_COMMAND is skipped.
p <ms>	Run a raw pulse. The shutter is first placed in a closed baseline and the minimum closed recovery guard is satisfied before opening for the requested command time.
p <ms> o	Run a compensated pulse. The requested time is treated as the target effective exposure and converted to a command time using the calibration lookup.
sw	Run the predefined raw timing sweep. In fast mode there is no five-second camera countdown, so begin camera recording before entering sw.
swo	Run the predefined compensated timing sweep. In fast mode there is no five-second camera countdown, so begin camera recording before entering swo.
ra / rb / rt	Disabled. Fast mode remains locked to Shutter A / NC and does not permit relay switching.
q	Quit with the shutter OPEN and the relay selector OFF.
qoff / qcoff	Quit OPEN / CLOSED with the relay selector OFF. Fast mode always leaves the selector OFF.

Fast mode timing and logging behavior

Fast mode uses a host-side closed-time guard rather than physical blade-position feedback. The default FAST_MIN_CLOSED_MS value is 200 ms. If the shutter was already commanded closed long enough, the next pulse does not add unnecessary waiting. Repeated close commands do not restart the recovery timer.

If a manual pulse is requested while the software believes the shutter is already open, the script warns that the recording may already contain extra exposure. Fast mode then closes the shutter, waits for the required recovery interval, performs the pulse, and closes again. Closing first cannot remove light that was already recorded.

A fast-mode manual command log is created before READY so file creation is kept out of the first pulse path. The log records pulse index, requested and commanded duration, whether compensation was used, host dispatch time, elapsed time, and command success. These timestamps are host-side dispatch times and are not direct measurements of the Arduino TTL edge or physical blade motion.

If a serial failure occurs in fast mode, the failure is latched and the script does not retry selector changes because the physical hardware state is no longer known. Inspect or reset the controller manually before continuing.

9. Timing and Flux Calibration

This is an experimental portion of the shutter system, and may change, this following section describes how the current shutter delays are captured and calculated.

The UFO shutter does not transition instantaneously between fully closed and fully open. Its blades retract from the center outward, so there is a finite mechanical delay between the first moment light passes through the opening and the moment the shutter is fully retracted. During opening, light first passes through the central portion of the aperture, and the transmitted flux increases as the clear aperture grows. Depending on the shutter's position in the optical path, this can produce either a mostly uniform increase in total throughput or a spatially dependent illumination pattern across the detector, where some regions of the sensor receive light before others. During closing, the reverse process occurs: the clear aperture shrinks over time, so the transmitted flux decreases gradually rather than stopping instantaneously. For short commanded exposures, especially commands shorter than the full mechanical open/close transition time, the delivered exposure is therefore better described by the integrated transmitted flux rather than by the electrical command duration alone.

The shutter is calibrated using high-speed video recordings of pulse sweeps using two cameras running at high fps at low resolution. The end goal is to convert a requested delivered exposure time into a command duration that delivers the same integrated flux.

Pulse sweep concept

The controller script can send predefined pulse trains for a camera to record. Current duration sets include:

```

durations1 =
[10,20,30,50,75,100,150,200,250,260,270,280,287,290,300,310,500,750,1000,1500
,2000,2500,3000,4000]
durations2 =
[10,12,15,17,20,22,24,26,28,30,32,34,36,38,40,42,45,50,60,75,80,85,100,150]
durations3 = [10,11,12,13,14,15,16,17,18,19,20,22,24,26,28,30]
durations4 = [75,76,77,78,79,80,81,82,83,84,85]
```

The current raw sweep gap is 3 seconds between pulses.

Flux metric

The updated flux analysis supports several metrics:

Mode	Purpose
APERTURE	Sum aperture flux while excluding saturated pixels
ANNULUS	Use a ring around the saturated core; useful for wings photometry
RING	Ring signal with separate outer background ring
TOPKCAP	Sum top-K pixels with saturation cap

For a star-like source with a saturated core, RING or ANNULUS is preferred. This avoids treating early core saturation as full mechanical retraction.

Effective exposure model

For each pulse, the normalized flux curve is integrated over the expected pulse window:

```
eff_ms = 1000 * sum(normalized_flux) / fps
```

This is interpreted as the flux-equivalent exposure time: the amount of fully-open exposure time that would produce the same integrated light.

The compensation model is then:

```
commanded_ms -> eff_ms
```

and the controller uses the inverse mapping:

```
target_eff_ms -> compensated_command_ms
```

Example analysis commands

Analyze one video with the default duration set:

```
python analyze_shutter_flux.py video.AVI --durations durations1 --gap-s 3.0  
--out-dir results_flux_new
```

Use ring/wings photometry:

```
python analyze_shutter_flux.py video.AVI \  
  --durations durations1 \  
  --gap-s 3.0 \  
  --mode RING \  
  --r-in 25 --r-out 60 \  
  --r-bg-in 75 --r-bg-out 105 \  
  --sat-thr 250 \  
  --out-dir results_flux_new
```

Plot the outputs:

```
python plot_shutter_flux.py --results-dir results_flux_new
```

10. Troubleshooting

Symptom	Likely cause	Suggested fix
Arduino disconnects, resets, or terminal says Device not configured	Relay noise, contact arcing, weak 5 V relay supply	Use external 5 V relay supply, add 680 μ F + 0.1 μ F caps, add snubbers, keep wiring short
Relay clicks but shutter does not move	Wrong COM/NO/NC wiring, no 24 V, wrong shutter selected	Verify relay contact wiring and UFO controller pins 7/8
Shutter opens when expected to close	TTL polarity inverted	Swap OPEN_STATE and CLOSED_STATE
Shutter A/B selection inverted	Relay module polarity or NO/NC wiring differs	Check RELAY_ON, RELAY_OFF, and contact terminal order
Dual shutter switches inconsistently	Switching while coil energized or relay noise	Use safe switch only; increase settle delays; verify snubbers
PyFirmata fails on Python 3.13	Deprecated <code>inspect.getargspec</code>	Patch to <code>inspect.getfullargspec</code> (see section 7)

11. Verification Procedure

Use this procedure after wiring changes.

1. Power everything off.
2. Verify controller terminal numbers before inserting wires.
3. Verify 24 V polarity to controller terminals 1 and 2.
4. Verify shutter coil wiring to terminals 7 and 8, or to relay COM/NC/NO in dual mode.
5. Verify Arduino D8 goes through 1 k Ω to terminal 10.
6. Verify Arduino GND goes to terminal 11.
7. Upload StandardFirmata to Arduino if board is unresponsive.
8. Connect Arduino only and run `ufo_shutter.py` without 24 V if testing logic.
9. Apply 24 V and test o and c on single shutter.
10. In dual mode, test ra and rb only through safe switching.
11. Run a long pulse such as `p 1000 n` to verify raw behavior.

12. Repository Layout

README.md	Main project README
ufo_shutter.py	Main pyFirmata shutter controller
ufo_shutter_test.ino.ino	Simple Arduino-only test sketch
analyze_shutter_avi.py	Earlier AVI timing analysis tool
analyze_shutter_flux.py	Flux / effective-exposure calibration tool
plot_shutter_flux.py	Plot calibration table and aligned flux
traces	
Current_WiringDiagram.png	Single-shutter wiring diagram
Dual_Shutter_Wiring.png	Dual-shutter relay wiring diagram
shutter_wires.png	LEMO / shutter connector wiring notes
original_wiring_diagram.png	Original FIREBall/GOBC wiring diagram
results_csv_timing/	Timing analysis results
results_csv_flux/	Flux analysis results
command_logs/	Runtime pulse command logs generated by
ufo_shutter.py	

13. Notes

- The Arduino controls only low-voltage logic: D8 for UFO controller TTL input and D9 for relay selection.
- Dual-shutter relay wiring works best with a separate 5 V relay supply, capacitors on the relay rails, and RC snubbers on relay contacts. Using a cheap 5V power supply may also introduce noise into the controller. Try using power adapter/usb cables with ferrite cores.
- The current compensation system prefers a monotone inverse lookup table over a single linear timing offset.
- Flux calibration should use annulus/ring/wings photometry when the source core saturates early.

14. TODO

- Rerun flux calibration with the final stable light source.
- Export a final master_inverse_lookup.csv for ufo_shutter.py.

Have questions? Contact @ ab5509@columbia.edu